

ACCIDENT PREDICTION&DETECTION BY EMBEDDED SENSOR

An Industry oriented Project Stage 1 Report Submitted to

Jawaharlal Nehru Technological University, Hyderabad

*In partial fulfillment of the requirements
for the award of the degree of*

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**

By

K.VIGHNESH (22E11A6692)

SK.KARIMULLA (23E15A6604)

Under the guidance of

Mrs. Syeda Hafsa Naaz

Assistant Professor Dept of CSE(AI&ML)

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**



BHARAT INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Approved by AICTE, Affiliated to JNTUH Hyderabad, NAAC Accredited, NBA Accredited
for UG Programmes CSE, ECE), Hyderabad - 501510

2025 - 2026



BHARAT INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Approved by AICTE, Affiliated to JNTUH Hyderabad, NAAC Accredited, NBA Accredited
for UG Programmes CSE, ECE), Hyderabad – 501510

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**

Certificate

*This is to certify that An Industry Oriented Mini project work entitled
“AI BASED ACCIDENT PREDICTION AND DETECTION USING EMBEDDED SENSORS” is
the bonafide work done*

	By	
K.VIGHNESH		(22E11A6692)
SK.KARIMULLA		(23E11A6604)

*In the Department of Computer Science and Engineering(AI&ML), **BHARAT INSTITUTE OF ENGINEERING AND TECHNOLOGY**, Hyderabad is submitted to **Jawaharlal Nehru Technological University, Hyderabad** in partial fulfillment of the requirements for the award of **B.Tech** degree in **Computer Science and Engineering(AI&ML)** during 2025-2026.*

Guide:

Mrs .Syeda Hafsa Naaz
Assistant Professor,
Dept. of CSE(AI&ML),
Bharat Institute of Engineering and Technology,
Hyderabad – 501 510.

Head of the Department:

Dr. Tawseef Ahmad
Asst Professor & Admin I/C,
Dept. of CSE(AI&ML),
Bharat Institute of Engineering and Technology,
Hyderabad – 501 510.

Viva-Voce held on.....

Internal Examiner

External Examiner

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success.

We avail this opportunity to express our deep sense of gratitude and hearty thanks to **Sri CH. Venugopal Reddy**, Secretary & Correspondent of BIET, for providing congenial atmosphere and encouragement.

We would like to thank **Prof. G. Kumaraswamy Rao, Director (R&D), Dr. V. Srinivas Rao, Dean CSE**, for having provided all the facilities and support.

We would like to thank **Dr. Tawseef Ahmad, Asst Professor & Admin I/C, Computer Science and Engineering (AI&ML)** for their expert guidance and encouragement at various levels of our Project.

We are thankful to our guide **Mr./Dr.Mrs.Syeda Hafsa Naaz, Assistant Professor, Computer Science and Engineering (AI*ML)** for his/her sustained inspiring Guidance and cooperation throughout the process of this project. His/her wise counsel and suggestions were invaluable. We are also thankful to our project coordinator **Mr. M. Suresh** for coordinating project at all levels for completion of the project.

We express our deep sense of gratitude and thanks to all the **Teaching and Non- Teaching Staff** of our college who stood with us during the project and helped us to make it a successful venture.

We place highest regards to our **Parents**, our **Friends** and **Well-wishers** who helped a lot in making the report of this project.

K.VIGHNESH	(22E11A6692)
SK.KARIMULLA	(23E11A6604)

DECLARATION

We hereby declare that this Industry oriented Project Stage 1 report is titled “***AI BASED ACCIDENT PREDICTION AND DETECTION USING EMBEDED SENSORS***” is a genuine project work carried out by us, in **B.Tech (Computer Science and Engineering-AI&ML)** degree course of **Jawaharlal Nehru Technology University Hyderabad, Hyderabad** and has not been submitted to any other course or university for the award of my degree by me.

Signature of the Student

K.VIGHNESH

SK.KARIMULLA

ACRONYMS AND DEFINITIONS

S No	ACRONYM	DESCRIPTION
1	AI	Artificial Intelligence- The stimulation of human intelligence in machines that are programmed to think and learn.
2	IoT	Internet of Things-Network of physical objects embedded with sensors,software and connectivity to exchange data.
3	ML	Machine Learning- A subset of AI that enables systems to learn from data and improve performance automatically.
4	GPS	Global Positioning System – A satellite-based navigation system used to determine the location of a vehicle or object.
5	GSM	Global System for Mobile Communication – A communication standard for transmitting data and messages over mobile networks.
6	IR	Infrared Sensor – A sensor used to detect obstacles, distance, or motion using infrared light.
7	GPU	Central Processing Unit – The main unit in the embedded system responsible for processing and executing commands.

ABSTRACT

Road accidents are one of the major causes of injuries and fatalities worldwide, leading to significant human and economic losses each year. To address this critical issue, this project proposes an AI-Based Accident Prediction and Detection System that utilizes embedded sensors and machine learning algorithms to predict, detect, and respond to accidents in real time. The system integrates various sensors such as accelerometers, gyroscopes, ultrasonic sensors, and GPS modules to continuously monitor the vehicle's motion, speed, and environmental conditions.

The collected sensor data is processed using artificial intelligence techniques to identify abnormal driving patterns that may lead to potential accidents. The prediction module employs supervised machine learning models trained on historical accident data to forecast possible collision risks. In the event of an actual accident, the detection module automatically activates an emergency response by sending the vehicle's location and status to nearby emergency services and registered contacts.

This combination of AI-driven analytics and embedded sensor technology enhances vehicle safety by providing early warnings and ensuring quick emergency response. The proposed system can be deployed in both modern and traditional vehicles with minimal hardware requirements. It demonstrates a cost-effective, intelligent solution for improving road safety, reducing accident impact, and saving lives through smart automation and real-time communication.

In the event of an actual collision, the accident detection module activates automatically. It verifies the impact based on sensor readings and immediately sends an alert message containing the vehicle's GPS coordinates and accident details to emergency services, nearby hospitals, and predefined emergency contacts. This rapid response mechanism significantly reduces the time required for assistance, thereby minimizing fatalities and severe injuries.

The system can be implemented using low-cost microcontrollers and sensors, making it suitable for both existing and new vehicles. It also supports IoT integration for cloud data storage and analysis, allowing continuous improvement of the predictive model. The combination of AI intelligence, embedded sensing, and IoT connectivity provides a powerful and cost-effective solution to enhance road safety.

In conclusion, the AI-Based Accident Prediction and Detection System demonstrates how emerging technologies can contribute to smarter and safer transportation. By predicting potential risks and providing instant detection and reporting capabilities, this project offers a promising approach to reducing accident-related losses and saving human lives.

CONTENTS

1.	Introduction	
1.1	Objective of Project	01
1.2	Limitations of project	01
2.	Literature Survey	
2.1	Introduction	03
2.2	Existing System	03
2.3	Disadvantages of Existing System	03
2.4	Proposed System	03
2.5	Advantages of Proposed System	04
3.	Methodology and Problem Identification	
3.1	User Requirement Specification	05
3.2	Software Requirement Specification	05
3.3	Hardware Requirement Specification	05
3.4	Content Diagram of project	06
4.	Design and Implementation	
4.1	UML/DFD/ER Diagram	07
4.2	Key Functions	08
4.3	Method of Implementation	10
4.4	Explanation of Key Functions	20
4.5	Method of Implementation	20
5.	Result Analysis	
5.1	Design of Test cases and scenarios	32
5.2	Test Results	34
6.	Conclusion	36
7.	References	37
	Annexure	39

INDEX FOR DIAGRAMS

Fig 1		06
Fig 2		10
Fig 3		10
Fig 4		11
Fig 5		11
Fig 6		13
Fig 7		13

1. INTRODUCTION

1. INTRODUCTION

1.1 Objective of Project:

The main objective of the project “AI-Based Accident Prediction and Detection Using Embedded Sensors” is to develop an intelligent and automated system that can accurately predict and detect road accidents in real time. This project aims to use artificial intelligence techniques along with embedded sensors to continuously monitor vehicle behavior, driving patterns, and surrounding environmental conditions. By analyzing sensor data and identifying abnormal or dangerous situations, the system strives to minimize accident risks and ensure enhanced safety for vehicle users.

Another key objective is to design a **low-cost and adaptable solution** that can be integrated into various vehicles without major modifications. The system should be capable of collecting data from sensors such as accelerometers, gyroscopes, vibration sensors, GPS modules, and temperature sensors. Using machine learning algorithms, the system will process this data to generate reliable predictions regarding potential accidents. In the case of a collision or sudden impact, the system will immediately detect the event and automatically send emergency alerts along with the vehicle’s exact location to predefined contacts or emergency services.

The project also aims to **reduce human error** by providing early warnings for situations such as overspeeding, rash driving, skidding, loss of control, or drowsiness. Through predictive analysis, the system intends to support safer driving practices and reduce fatal accident rates. Additionally, the project promotes the concept of smart transportation by combining AI, sensor networks, and communication technologies to create an efficient, autonomous, and scalable road safety solution.

Overall, the objective is to build a **smart accident prediction and detection system** that enhances driver safety, reduces emergency response time, and contributes to a safer transportation environment.

1.2 Limitations of project:

1.2.1 Code Limitations:

Although the proposed AI-Based Accident Prediction and Detection System offers several advantages, the code used in the project has certain limitations that affect its overall performance and reliability. One major limitation is the dependency on the quality and quantity of the training data. Since the machine learning model relies heavily on sensor datasets to make accurate predictions, insufficient or unbalanced data may lead to misclassification or incorrect alerts. The system may also struggle to identify rare accident scenarios that were not included in the training dataset, limiting its predictive accuracy under unpredictable real-world conditions.

Another limitation of the code is its sensitivity to sensor noise and hardware inconsistencies. Embedded sensors such as accelerometers, gyroscopes, and vibration modules often generate fluctuating or noisy signals, which can affect the stability of the code's output. Although filtering techniques are applied, the code may still misinterpret certain noise patterns as emergency situations. Additionally, variations in vehicle type, road conditions, or mounting positions of sensors can cause inconsistencies in the data, leading to unexpected behavior or false triggering of the system.

The processing capabilities of embedded hardware also impose constraints on the code. Since microcontrollers have limited memory and computational power, the system cannot run highly complex algorithms or large machine learning models. As a result, the code must be simplified, which may reduce the accuracy or responsiveness of the system. Real-time data handling becomes more challenging when multiple sensors generate continuous streams of data, and the microcontroller may sometimes experience delays or missed readings.

Another limitation arises from communication constraints. The code depends on communication modules such as GSM, GPS, or IoT platforms to send alerts, and any network failure may prevent timely message delivery. In remote areas with poor signal strength, the system may not function as intended despite the code running correctly.

Overall, while the code forms the core of the accident prediction and detection system, its functionality is limited by factors such as data quality, hardware constraints, environmental variability, and communication reliability. Future improvements can focus on optimizing algorithms, increasing dataset diversity, and integrating more robust sensor calibration techniques to enhance overall system performance.

1.2.2. Procedures, Types, and Variables:

The development of the AI-Based Accident Prediction and Detection System involves a systematic set of procedures, well-defined types of components, and clearly identified variables that play a major role in the functioning of the system. The procedures outline the step-by-step methodology followed from data collection to real-time prediction. The types describe the classification of hardware, software, algorithms, and sensors used. The variables determine the values monitored by the system to detect unusual vehicle behavior or potential accident conditions. Together, these elements ensure proper system design, consistent performance, and accurate accident prediction.

1. Procedures:

The overall procedure followed in the project can be divided into several phases. Each phase represents a crucial stage in developing and implementing the accident prediction and detection mechanism.

a) Data Collection Procedure

The first procedure involves collecting raw sensor data from accelerometers, gyroscopes, temperature sensors, vibration sensors, and GPS modules. The data is recorded under various driving conditions such as normal driving, sudden braking, sharp turning, skidding, and collision tests.

b) Data Preprocessing Procedure

The next procedure is cleaning and preprocessing the collected data. Noise removal, filtering, normalization, and sensor calibration are performed to ensure the data is suitable for machine learning analysis. Missing values are corrected, and irrelevant data is removed to improve model accuracy.

c) Feature Extraction Procedure

From the cleaned data, important features are extracted. These include acceleration spikes, angular rotation, tilt angle, sudden vibrations, speed variations, and location changes. These features are essential for training the AI model.

d) Model Training Procedure

Machine learning algorithms such as Random Forest, KNN, or ANN are trained using the extracted features. Training involves feeding input data with labeled accident and non-accident cases so the model can learn patterns.

e) Real-Time Monitoring Procedure

After training, the model is deployed in the embedded system. The microcontroller continuously reads data from sensors and processes it to detect abnormal patterns indicating a potential accident.

f) Accident Prediction Procedure

The AI model compares real-time sensor readings with learned patterns. If the readings cross predefined thresholds or match accident-like behavior, the system predicts a possible accident and alerts the driver.

g) Accident Detection and Notification Procedure

In case of a collision, the system triggers an emergency alert. A message containing GPS coordinates is automatically sent to registered contacts or emergency services through GSM or IoT modules.

2. Types

The project uses different types of sensors, data processing methods, algorithms, and communication technologies. These components are classified based on their functionality.

a) Types of Sensors

- **Accelerometer:** Measures linear acceleration and detects sudden impact.
- **Gyroscope:** Measures angular rotation and tilt for accident identification.
- **Vibration Sensor:** Detects abrupt vibrations during collision.
- **Temperature Sensor:** Monitors changes in vehicle engine temperature.
- **GPS Module:** Provides real-time location tracking.

b) Types of Algorithms

- **Classification Algorithms:** Random Forest, Decision Tree, KNN.
- **Sequential Models:** Neural Networks for pattern recognition.
- **Threshold-Based Rules:** Predefined limits for detecting impacts.

c) Types of Communication Modules

- **GSM Module:** Sends SMS alerts in case of emergency.

- **IoT Module (ESP8266/ESP32):** Cloud-based monitoring and notification.

d) Types of Data

- **Training Data:** Sensor readings with labels.
- **Testing Data:** Used to evaluate model accuracy.
- **Real-Time Data:** Continuous vehicle monitoring data.

e) Types of Responses

- **Predictive Alerts:** Warning message before accident.
- **Emergency Alerts:** Automated message after collision.
- **Data Logging:** Storing sensor data for analysis.

3. Variables

Variables play a critical role in determining system behavior. These variables capture environmental conditions, vehicle movement, and driver actions.

a) Sensor Variables

- **Acceleration (A_x, A_y, A_z):** Sudden spikes indicate impact.
- **Angular Rotation (G_x, G_y, G_z):** High values indicate rollover or sharp turns.
- **Vibration Intensity:** Used for collision detection.
- **Vehicle Speed (V):** High or inconsistent speed indicates risk.
- **Temperature Level:** Abnormal levels indicate engine malfunction.

b) Environmental Variables

- **Road Surface Condition:** Affects sensor readings during skidding.
- **Curvature of Road:** Influences gyroscope readings.

- **Weather Conditions:** May cause sensor noise.

c) Threshold Variables

These variables act as limits to trigger alerts.

- **Acceleration Threshold:** Value beyond which impact is suspected.
- **Rotation Threshold:** Set to detect vehicle rollover.
- **Speed Threshold:** To identify overspeeding conditions.

d) Machine Learning Variables

- **Input Features:** Sensor readings used to train the model.
- **Output Labels:** Accident / Non-Accident classification.
- **Accuracy Metrics:** Used to evaluate model performance.

e) Communication Variables

- **Latitude and Longitude:** Sent during emergency alert.
- **Network Strength:** Affects alert transmission time.
- **Device ID:** Identifies the specific vehicle unit.

1.2.3 Runtime limitations:

The AI-Based Accident Prediction and Detection System faces several runtime limitations that affect its performance during real-time execution. Since the system relies on continuous data monitoring and fast processing, any delay or hardware constraint can reduce its accuracy. One major runtime limitation is the restricted processing power of embedded microcontrollers. These devices have limited CPU speed and memory, which can slow down the execution of machine learning algorithms. As a result, complex models cannot be executed smoothly in real time, and the system may experience lag or delayed response during critical moments.

Another runtime limitation arises from sensor data fluctuations. During real-world operation, sensors such as accelerometers, gyroscopes, and vibration modules produce rapidly changing readings. These high-

frequency inputs require quick sampling and immediate analysis. If the microcontroller fails to handle this data load efficiently, it may skip some readings or process outdated values, leading to inaccurate accident predictions. This is especially problematic during high-speed travel, where even a small delay of milliseconds can affect detection performance.

Communication delays also contribute to runtime limitations. Modules like GSM, GPS, and IoT-based communication often face signal fluctuations, low bandwidth, or temporary network loss. These communication challenges can slow down the alert-sending process or completely interrupt emergency notifications. In remote or rural areas, poor network connectivity increases the runtime delay significantly, affecting the system's reliability during actual accidents.

Power consumption and battery limitations can further impact runtime performance. Continuous sensor monitoring and active communication require sufficient power. If the power supply is unstable or the battery drains quickly, the system may shut down or operate inconsistently. This can disrupt real-time monitoring and reduce the overall effectiveness of the system.

Another critical runtime limitation is environmental interference. Vibrations, road bumps, electromagnetic noise, and temperature changes can cause the sensors to behave unpredictably during operation. These conditions may lead to runtime errors, false-positive alerts, or incorrect readings. The system must filter and correct this data instantly, which puts additional load on the processor.

Overall, runtime limitations such as computational delays, sensor instability, communication issues, power constraints, and environmental interference affect the real-time behavior of the accident prediction and detection system. Addressing these limitations requires optimizing the code, using efficient algorithms, reducing processing overhead, and selecting higher-performance embedded hardware to ensure reliable real-time operation.

In addition to the earlier challenges, the system also faces runtime limitations due to the dependency on continuous sensor calibration. During real-time usage, sensors can gradually drift from their original calibration values due to mechanical vibration, temperature changes, or aging of components. If the system does not recalibrate periodically, the accuracy of accident prediction decreases over time. This sensor drift may cause the AI model to misinterpret normal driving patterns as dangerous situations or fail to detect sudden impacts correctly.

Another major runtime limitation is the dependency on data synchronization. In this system, multiple sensors work together and produce data at different intervals. Ensuring proper synchronization between accelerometer data, gyroscope readings, GPS updates, and vibration sensor output is crucial for accurate real-time prediction. If there is even a slight mismatch in timing, the machine learning algorithm may receive incomplete or misaligned inputs, leading to incorrect decisions. Achieving perfect synchronization on low-cost microcontrollers is challenging, which creates delays and inconsistencies during runtime.

Memory overflow is another limitation during continuous execution. As real-time monitoring generates large volumes of data, the system may run out of buffer space if the microcontroller does not clear memory efficiently. This may cause system slowdowns, freezing, or unexpected resets. Since embedded devices have limited RAM, improper memory management becomes a critical runtime challenge. Storing temporary variables, processed features, and communication data can strain memory resources.

The system is also affected by firmware bugs or unexpected software behavior. During long periods of continuous operation, minor coding errors may cause stack overflow, infinite loops, or unexpected crashes. Runtime errors can disrupt the accident detection process and may leave the system unresponsive during critical moments. Since the device is expected to run for long durations without manual supervision, robustness against runtime bugs is essential.

Vehicle-specific variations also create runtime limitations. Different vehicles produce different vibration patterns, movement signatures, and acceleration responses. A model trained for one type of vehicle may not perform accurately in another unless the system is tuned or retrained. As a result, runtime accuracy decreases when the system is installed in a new vehicle without calibration.

Finally, extreme environmental conditions such as heavy rain, fog, high heat, or muddy roads can influence sensor performance. Water droplets, dust, or debris may affect sensor readings and cause runtime distortions. The system must compensate for these environmental fluctuations, but low-cost embedded hardware struggles to handle such complex corrections in real time.

Overall, these additional runtime limitations highlight the challenges of implementing a continuous, AI-driven vehicle safety system on embedded hardware. Overcoming these issues requires more powerful processors, improved calibration methods, better firmware optimization, and adaptive machine learning models to ensure consistent and reliable real-time performance.

1.2.4 Unsupported Web.config elements:

When building an AI-based accident prediction and detection project that includes a web or cloud component, some traditional ASP.NET web.config elements may be unsupported, ignored, or unsuitable depending on the target runtime (for example: .NET Core / .NET 5+, Azure App Service, or modern hosting where appsettings.json and host-managed settings are preferred). Below is a practical one-page summary of common web.config elements you should avoid or replace, why they are problematic for this project, and suggested alternatives.

1. <system.web> legacy sections (e.g., <httpModules>, <httpHandlers>)

These are from classic ASP.NET and are not used in ASP.NET Core. In hosted environments that run Core, these sections are ignored. Use middleware registration in Startup.cs (or in Program.cs) instead.

2. <membership>, <roleManager>, <profile>

Legacy membership/role providers are not supported in Core and are insecure/outdated for modern identity needs. Use ASP.NET Core Identity, or external providers (Azure AD, Auth0, Firebase Auth) for authentication and role management.

3. <identity> / <trust>

Some hosts (shared hosting, Azure App Service) disallow changing process identity or trust levels. These elements may be blocked or cause deployment errors. Rely on host security settings and managed identities (Azure Managed Identity) instead.

4. <machineKey>

In many cloud or load-balanced setups the machine key must be managed centrally. Some platforms override or disallow explicit machineKey settings. Use platform features (sticky sessions or distributed data protection keys) or ASP.NET Core Data Protection with a shared key store (Azure Blob, Redis).

5. <healthMonitoring> / <profilePerformanceCounter>

These Windows/IIS specific monitoring elements are often

unsupported in containerized or cross-platform environments. Use application insights, Prometheus, or other platform telemetry instead.

6. <httpRuntime> advanced attributes

Some attributes (e.g., requestValidationMode, maxRequestLength) behave differently or are ignored in Core. Configure request limits and validation in the hosting server (Kestrel) or reverse proxy settings.

7. <customErrors> vs. modern error handling

While customErrors works in classic IIS, in Core use exception handling middleware (UseExceptionHandler) and centralized error pages / logging.

2.LITERATURE SURVEY

2. LITERATURE SURVEY

2.1 Introduction:

The rapid increase in road accidents across the world has created a strong demand for intelligent systems capable of predicting and detecting accidents in real time. Traditional road safety mechanisms rely heavily on human response, manual monitoring, and post-accident reporting, which often leads to delayed assistance and increased fatalities. With the advancements in Artificial Intelligence (AI), Machine Learning (ML), and embedded sensor technologies, researchers have explored new methods that can automatically analyze vehicle behavior and environmental conditions to prevent accidents before they occur. This literature survey provides a comprehensive background of the research conducted in this field, highlighting technological developments, existing methodologies, and the limitations that have motivated the creation of modern AI-based accident prediction and detection systems.

In recent years, the combination of embedded sensors with machine learning algorithms has gained significant attention because of its capability to continuously monitor vehicle movement and interpret patterns that represent risky driving conditions. Various studies emphasize the integration of accelerometers, gyroscopes, GPS modules, and vibration sensors to collect real-time data from vehicles. This data is often processed using predictive algorithms such as Decision Trees, Random Forests, Support Vector Machines (SVM), and Neural Networks to classify driving behavior and detect possible accidents. The literature shows that such AI-driven systems offer faster response time, higher accuracy, and better adaptability compared to traditional mechanical safety systems. However, many research articles also highlight challenges such as sensor noise, environmental interference, limited computational power, and communication delays that affect system performance in real-world situations.

Another essential focus of existing literature is the use of Internet of Things (IoT) platforms for emergency communication. Several

researchers propose systems where accident alerts, along with GPS coordinates, are automatically sent to emergency contacts, hospitals, or traffic authorities. Studies show that the combination of AI for prediction and IoT for communication significantly reduces the time between the accident and medical response, potentially saving lives. Literature also discusses the role of cloud computing and mobile applications in providing live monitoring, centralized data storage, and advanced analytics that support transportation authorities and vehicle manufacturers in improving road safety management.

The literature further discusses the advantages and limitations of on-board vehicle systems compared to smartphone-based solutions. Some studies propose using smartphones for accident detection since they already contain built-in sensors such as accelerometers and GPS modules. However, research also identifies issues such as battery drain, inconsistent placement of the phone, network dependency, and user behavior, which limit the reliability of smartphone-based systems. Embedded hardware, on the other hand, offers more accurate sensor readings and can be permanently installed in any vehicle. Because of this, many authors recommend microcontroller-based AI systems as more stable and suitable for continuous monitoring.

Overall, the literature highlights a clear shift from traditional accident detection methods to intelligent, automated, and real-time safety analysis systems powered by AI and sensors. Despite significant progress, existing research identifies several gaps, such as limited datasets, lack of system calibration for different vehicle types, and challenges in executing complex AI models on low-power hardware. These limitations present an opportunity for further innovation and refinement. The present project aims to address some of these issues by developing a practical, low-cost, and efficient AI-based accident prediction and detection system using embedded sensors, offering improved accuracy and reliability for real-time accident prevention.

2.2 Existing System:

The existing systems developed for accident detection and road safety primarily rely on conventional mechanical sensors, manual reporting methods, or smartphone-based monitoring solutions. These systems provide partial assistance in identifying accidents but lack predictive capabilities and real-time automated responses. Most traditional accident detection systems activate only after a collision has occurred, and they often depend on the driver or passengers to manually report the incident. This leads to delays in medical support, especially in situations where the passengers are unconscious or where the accident occurs in remote locations with poor connectivity. Existing literature repeatedly emphasizes that such traditional methods are insufficient for today's fast-moving traffic conditions, where rapid response plays a crucial role in saving lives.

One widely used existing system is the airbag deployment sensor, which relies on the detection of strong impacts to trigger the airbags and, in some advanced vehicles, to send limited information to manufacturer servers. However, these systems only respond after the accident, not before, and are restricted to high-end vehicles. Literature also highlights the limitations of systems such as OnStar and similar telematics services, which depend on expensive hardware and subscription models. These systems can provide emergency response services, but they are inaccessible to most low-cost or older vehicles. Additionally, these systems do not analyze driving patterns or environmental conditions, meaning they cannot predict accidents in advance.

Another category of existing systems uses smartphone-based accident detection applications. These mobile apps use the phone's accelerometer, gyroscope, and GPS to detect sudden impacts or abnormal movements. While these systems are low-cost and easy to install, research studies identify major limitations: phones can be placed anywhere in the vehicle, causing inaccurate readings; battery drainage affects performance; network dependency causes delays; and user intervention is often needed. The inconsistent orientation of smartphones

leads to unreliable detection and produces false alarms in many real-world scenarios. Literature strongly suggests that smartphone-only solutions are unstable and cannot fully replace dedicated embedded vehicle hardware.

Existing systems also include simple sensor-based devices installed in vehicles. These may use accelerometers or vibration sensors to detect crashes. They can automatically send a message to emergency contacts, but they lack intelligence. Most of these devices rely on fixed threshold values to detect impact, which leads to two major problems:

1. False Positives – Normal vibrations from rough roads can cross the threshold, triggering false accident alerts.
2. False Negatives – Low-intensity but critical accidents may not cross the threshold, causing the system to fail in detection.

These threshold-based systems cannot differentiate between normal driving irregularities and actual pre-accident scenarios, making them less reliable in real-world conditions.

Many existing systems also lack predictive capabilities. They only detect accidents after impact rather than forecasting risky conditions beforehand. Literature points out that road accidents are often preceded by patterns such as overspeeding, harsh braking, sharp turns, lane drifting, skidding, or driver drowsiness. Traditional systems do not analyze these patterns because they lack machine learning algorithms and real-time data processing ability. This creates a significant technological gap that motivates the development of AI-based systems capable of prediction as well as detection.

Another key limitation of existing systems highlighted in research is their poor connectivity with emergency services. Even if a crash is detected, the alert often does not reach relevant authorities in time due to network issues, lack of automated communication modules, or improper integration with IoT platforms. Several studies indicate that delays in communication remain one of the leading causes of fatalities, as quick response is essential for survival in critical injuries.

In summary, the existing systems reviewed in literature provide foundational mechanisms for accident detection but suffer from limitations such as lack of predictive intelligence, dependence on threshold-based logic, high cost, poor reliability, user dependency, hardware restrictions, and communication delays. These shortcomings highlight the need for advanced AI-integrated embedded systems that can monitor vehicle behavior continuously, analyze sensor data intelligently, predict accidents before they occur, and ensure immediate automated emergency communication. The present project aims to overcome these limitations by designing a smart, low-cost, and efficient AI-based accident prediction and detection system using embedded sensors.

2.3 Disadvantages of Existing System:

The existing accident detection systems used in vehicles suffer from several disadvantages that limit their effectiveness in real-time accident management and road safety. One major disadvantage is that most traditional systems are reactive rather than predictive. They can detect an accident only after it has already occurred, providing no warning or preventive support to the driver. As a result, they fail to reduce accident risks or prevent collisions in advance. This major limitation highlights the need for intelligent AI-based models capable of analyzing driving patterns and predicting accidents before they happen.

Another significant drawback of existing systems is their dependence on fixed threshold values. Systems using accelerometers or vibration sensors often classify accidents based on predefined limits. This approach is unreliable because normal road bumps, potholes, or sudden brakes may trigger false alarms, while minor yet dangerous collisions may not cross the threshold and go undetected. This leads to both false positives and false negatives, reducing the credibility and usefulness of such systems.

Smartphone-based accident detection applications also present several disadvantages. Their accuracy depends heavily on phone position, battery level, network connectivity, and user behavior. If the phone is placed loosely, turned off, or low on battery, the system becomes ineffective. Additionally, smartphones generate inconsistent sensor readings due to their varying orientations, making them unsuitable for reliable detection in critical situations. Literature shows that phone-based systems often fail during high-speed or impact-heavy scenarios. Existing vehicle-based emergency alert systems are also limited by communication delays. Many systems rely on GSM or manual reporting, which can be slow, especially in areas with poor network coverage. In several real-world cases, emergency services receive delayed information, reducing the chances of timely medical assistance. Furthermore, high-end telematics systems such as OnStar are expensive and not accessible to low-cost vehicles, making them

unsuitable for widespread use.

Another key disadvantage is the lack of integration and intelligence in older systems. They cannot process data from multiple sensors or analyze complex driving conditions. They operate independently without using machine learning, artificial intelligence, or data fusion methods. As a result, they cannot distinguish between normal and hazardous driving patterns. These systems also do not adapt to different vehicle types, driver behaviors, or environmental conditions. Overall, the existing systems lack prediction capability, suffer from threshold-based limitations, face communication delays, depend on user behavior, and are often expensive or inaccessible. These disadvantages clearly show the need for an advanced, affordable, and intelligent AI-based accident prediction and detection system capable of providing accurate real-time monitoring and automated emergency response.

2.4 Proposed System:

The proposed system introduces an intelligent, AI-driven Accident Prediction and Detection System using embedded sensors, addressing the major limitations found in existing road safety technologies.

Unlike traditional systems that detect accidents only after collision, the proposed system integrates advanced machine learning algorithms to continuously monitor vehicle behavior, analyze real-time sensor data, and predict potential accidents before they occur. This predictive capability helps reduce accidents caused by overspeeding, harsh braking, sharp turns, skidding, or driver inattention. By using a combination of accelerometer, gyroscope, vibration sensor, GPS module, and a microcontroller, the system ensures accurate analysis of vehicle dynamics and instant response to unsafe conditions.

The proposed system consists of three major components: accident prediction, accident detection, and automated emergency alert. The prediction module uses AI algorithms trained on historical sensor datasets to identify patterns linked to dangerous driving behaviors. If abnormal patterns are observed in the real-time data stream, the system immediately issues a warning to the driver, helping them correct their driving and avoid a potential accident. This proactive monitoring drastically reduces the probability of collision. The accident detection module, on the other hand, activates only when an actual impact or sudden force is detected. This ensures high accuracy by combining predictive analysis with impact-based confirmation.

Another major improvement in the proposed system is the integration of IoT-based communication for emergency alerts. When an accident is detected, the system automatically sends a message with the vehicle's GPS coordinates to predefined emergency contacts or authorities. This eliminates delays caused by manual reporting, especially in cases where the driver is unconscious or unable to use a phone. The system can also upload the data to a cloud server for real-time monitoring and analysis. This ensures faster response time from medical teams and helps reduce fatalities caused by delayed

assistance.

The proposed system also addresses the disadvantages of smartphone-based solutions by using dedicated embedded sensors mounted securely inside the vehicle. These sensors provide more stable, consistent, and accurate readings compared to mobile devices. The microcontroller processes the data locally, reducing dependency on external devices and ensuring that the system remains operational even when network connectivity is weak. Furthermore, the system is designed to be low-cost and compatible with all vehicle types, making it suitable for mass adoption in developing regions where advanced telematics systems are too expensive.

In addition to prediction and detection, the proposed system includes improved data fusion techniques that combine readings from multiple sensors to reduce noise and false alarms. Machine learning models further enhance accuracy by adapting to different driving styles, vehicle structures, and road conditions. This adaptive feature makes the proposed system more reliable than fixed-threshold methods used in traditional systems. The system also supports real-time calibration, allowing it to adjust its sensitivity based on environmental factors such as road roughness, vehicle load, and weather conditions.

Overall, the proposed system brings a modern, intelligent, and highly responsive solution to the challenge of road accident prevention. By integrating AI, IoT, and embedded hardware, the system provides continuous monitoring, real-time prediction, automatic emergency alerts, and adaptive learning capabilities. It is designed to overcome the limitations of existing systems, reduce accident rates, enhance driver safety, and ensure timely medical intervention. With its low cost, high accuracy, and scalability, the proposed system offers a practical and effective approach to improving road safety in today's fast-moving transportation environment.

2.5 Advantages of Proposed System :

The proposed AI-Based Accident Prediction and Detection System offers several advantages over existing traditional and smartphone-dependent methods. One of the major advantages is its ability to predict accidents before they occur, unlike conventional systems that only detect accidents after impact. By continuously analyzing real-time sensor data using machine learning models, the system identifies abnormal patterns such as overspeeding, sudden sharp turns, harsh braking, or skidding. When the system predicts a possible risk, it immediately alerts the driver, thereby reducing the chances of collision. This predictive capability is a significant improvement over fixed-threshold systems, which cannot analyze complex driving behavior or detect pre-accident conditions.

Another key advantage of the proposed system is its high accuracy and reliability, achieved through the integration of multiple embedded sensors such as accelerometers, gyroscopes, vibration sensors, and GPS modules. The combined readings from these sensors provide a more complete understanding of vehicle dynamics. Machine learning algorithms process the data to distinguish between normal driving variations and dangerous activities, reducing false positives and false negatives. This multi-sensor and AI-driven approach ensures real-time detection with greater precision compared to older systems that depend on simple mechanical sensors or single-parameter monitoring.

A major advantage of the proposed system is its automated emergency alert feature. If a serious impact or crash occurs, the system immediately sends an alert message along with the vehicle's GPS location to the registered emergency contacts or nearby services. This eliminates the need for human intervention during critical moments, particularly when the driver is unconscious or unable to make a call. Faster communication increases the chances of survival by ensuring timely medical response. In addition, the system can upload sensor data to a cloud server for real-time monitoring by authorities, making emergency handling more efficient.

The proposed system is designed to be cost-effective, scalable, and

compatible with all types of vehicles, including two-wheelers, four-wheelers, commercial vehicles, and older models that lack modern safety features. Unlike expensive telematics systems available only in high-end vehicles, this system uses affordable embedded hardware and open-source machine learning technologies, making it suitable for widespread adoption, especially in developing countries. Its low-cost design does not compromise performance, making advanced accident prediction technology accessible to a larger population.

Another advantage is the system's ability to operate independently of smartphones, addressing the limitations of mobile-based solutions such as inconsistent sensor accuracy, battery drain, unstable placement, and network dependency. Since the proposed system uses dedicated hardware installed permanently in the vehicle, it delivers stability, consistency, and dependable performance. The embedded sensors are not influenced by user behavior, phone orientation, or device limitations, making the system more trustworthy in real-time accident scenarios.

The system also benefits from adaptive learning, where the AI model continuously improves its accuracy by learning from new driving patterns, environmental changes, and vehicle-specific characteristics. Unlike fixed-threshold methods, the proposed system evolves over time and becomes better at distinguishing between normal and risky behaviors. This intelligent adaptability ensures that the system performs reliably across different terrains, road conditions, and weather environments.

Furthermore, the proposed system enhances overall road safety by promoting safe driving practices. The real-time alerts help drivers reduce overspeeding, maintain lane discipline, and avoid sudden risky maneuvers. This not only prevents accidents but also encourages long-term responsible driving behavior. Transportation authorities and fleet management companies can also use the system to monitor driver performance and improve safety regulations.

In summary, the proposed AI-based system offers numerous advantages, including accident prediction capability, high accuracy

through sensor fusion, automated emergency communication, low cost, adaptability, independence from smartphones, and enhanced driver safety. These advantages make the system a powerful and practical solution for reducing road accidents, preventing fatalities, and improving the efficiency of emergency response mechanisms. By overcoming the limitations of existing systems, the proposed design provides a reliable, intelligent, and scalable approach to modern vehicle safety

3.METHODOLOGY

3. METHODOLOGY AND PROBLEM IDENTIFICATION

3.1 User Requirement :

1. Arduino IDE / PlatformIO

- Used for writing, compiling, and uploading code to microcontrollers.
- Supports libraries for accelerometer, gyroscope, GPS, GSM, Wi-Fi, and other sensors.

2. Python Programming Environment

- Required for training machine learning models.
- Used for preprocessing datasets and performing analysis.

3. Jupyter Notebook / Google Colab

- Platforms for running ML experiments, visualizing data, and testing algorithms.
- GPU/TPU support helps in faster training.

4. Machine Learning Libraries

- NumPy – numerical computations.
- Pandas – data handling and processing.
- Scikit-Learn – training ML models like Decision Trees, KNN, Random Forest, etc.
- TensorFlow / Keras – neural network model creation.

5. Sensor Libraries

- Libraries for Accelerometer (ADXL335/MPU6050).
- Libraries for Gyroscope, GPS (NEO-6M), and Vibration sensors.
- Ensure accurate real-time readings.

6. Data Preprocessing & Filtering Algorithms

- Noise filtering (Kalman filter, Moving Average).
- Normalization and smoothing algorithms.
- Sensor calibration scripts.

7. Real-Time Processing Software Modules

- Interrupt-based code for continuous sensor data reading.
- Fast loop execution for real-time monitoring.
- Timestamping and data synchronization.

8. Communication Protocol Libraries

- GSM libraries for sending SMS alerts.
- Wi-Fi / MQTT libraries for IoT-based notifications.
- APIs for sending GPS coordinates.

9. Cloud / IoT Platforms (if used)

- ThingSpeak, Blynk, Firebase, Ubidots, or Adafruit IO.
- Used for dashboard creation and remote monitoring.

10. Mobile / Web App Tools (optional)

- HTML, CSS, JavaScript for web dashboard.
- Android Studio / MIT App Inventor for mobile alert apps.

11. Serial Monitor / Debugging Tools

- Built-in Arduino Serial Monitor.
- Real-time debugging during sensor testing.

12. Version Control Tools

- Git / GitHub for storing source code.
- Tracks software changes and supports teamwork.

13. Model Conversion Tools

- TensorFlow Lite Converter (if deploying ML model to microcontroller).
- Ensures smaller model size and fast inference.

14. Firmware Update Support Tools

- OTA (Over-The-Air) update libraries (for ESP32/ESP8266).
- Allows software updates without wired connection.

3.2 Software Requirement :

1. Arduino IDE / PlatformIO

- Used for writing, compiling, and uploading code to microcontrollers.
- Supports libraries for accelerometer, gyroscope, GPS, GSM, Wi-Fi, and other sensors.

2. Python Programming Environment

- Required for training machine learning models.
- Used for preprocessing datasets and performing analysis.

3. Jupyter Notebook / Google Colab

- Platforms for running ML experiments, visualizing data, and testing algorithms.
- GPU/TPU support helps in faster training.

4. Machine Learning Libraries

- NumPy – numerical computations.
- Pandas – data handling and processing.
- Scikit-Learn – training ML models like Decision Trees, KNN, Random Forest, etc.
- TensorFlow / Keras – neural network model creation.

5. Sensor Libraries

- Libraries for Accelerometer (ADXL335/MPU6050).
- Libraries for Gyroscope, GPS (NEO-6M), and Vibration sensors.
- Ensure accurate real-time readings.

6. Data Preprocessing & Filtering Algorithms

- Noise filtering (Kalman filter, Moving Average).

- Normalization and smoothing algorithms.
- Sensor calibration scripts.

7. Real-Time Processing Software Modules

- Interrupt-based code for continuous sensor data reading.
- Fast loop execution for real-time monitoring.
- Timestamping and data synchronization.

8. Communication Protocol Libraries

- GSM libraries for sending SMS alerts.
- Wi-Fi / MQTT libraries for IoT-based notifications.
- APIs for sending GPS coordinates.

9. Cloud / IoT Platforms (if used)

- Thingspeak, Blynk, Firebase, Ubidots, or Adafruit IO.
- Used for dashboard creation and remote monitoring.

10. Mobile / Web App Tools (optional)

- HTML, CSS, JavaScript for web dashboard.
- Android Studio / MIT App Inventor for mobile alert apps.

11. Serial Monitor / Debugging Tools

- Built-in Arduino Serial Monitor.
- Real-time debugging during sensor testing.

12. Version Control Tools

- Git / GitHub for storing source code.
- Tracks software changes and supports teamwork.

13. Model Conversion Tools

- TensorFlow Lite Converter (if deploying ML model to microcontroller).

- Ensures smaller model size and fast inference.

14. Firmware Update Support Tools

- OTA (Over-The-Air) update libraries (for ESP32/ESP8266).
- Allows software updates without wired connection.

3.3 Hardware Requirement :

1. Microcontroller / Development Board

- Arduino Uno / Arduino Mega
- ESP32 / NodeMCU (recommended for IoT + faster processing)
- Acts as the central processing unit of the system.

2. Accelerometer Sensor

- ADXL335 or MPU6050
- Measures linear acceleration and detects sudden impacts.

3. Gyroscope Sensor

- MPU6050 (combined accelerometer + gyroscope)
- Detects angular rotation, tilt, and sudden direction changes.

4. Vibration Sensor

- SW-420 or Piezoelectric vibration sensor
- Detects collision vibrations or sudden shocks.

5. GPS Module

- NEO-6M GPS Module
- Provides accurate location coordinates for emergency alerts.

6. GSM Module (for SMS alerts)

- SIM800L or SIM900A
- Sends emergency messages to registered contacts.

7. IoT Module (optional)

- ESP8266 / ESP32 Wi-Fi
- Used for real-time cloud reporting and online dashboards.

8. Power Supply Unit

- 5V/9V power supply
- Vehicle battery connection / USB power
- Ensures stable operation of all sensors.

9. Voltage Regulator

- LM7805 or buck converter
- Maintains stable voltage for sensitive modules.

10. Connecting Wires / Jumper Cables

- For connecting sensors to the microcontroller.

- Ensures stable signal transmission.

11. Breadboard / PCB

- Breadboard for testing and prototyping.
- PCB for final stable installation in the vehicle.

12. Buzzer / Alarm Module

- Provides alert sounds for pre-accident warnings.

13. LED Indicators

- Indicates system status, sensor activity, and alert triggering.

14. Enclosure / Casing Box

- Protects all components from dust, heat, and vibration inside the vehicle.

15. Smartphone (Optional)

- Used only if mobile app monitoring is included.
- Not required for core system operation.

16. Laptop / PC

- Required for programming the microcontroller.
- Used for model training, data analysis, and debugging.

17. Backup Battery (Optional)

- Small Li-ion battery for uninterrupted operation.
- Keeps the system active even if vehicle power fails.

3.4 Content diagram of project :

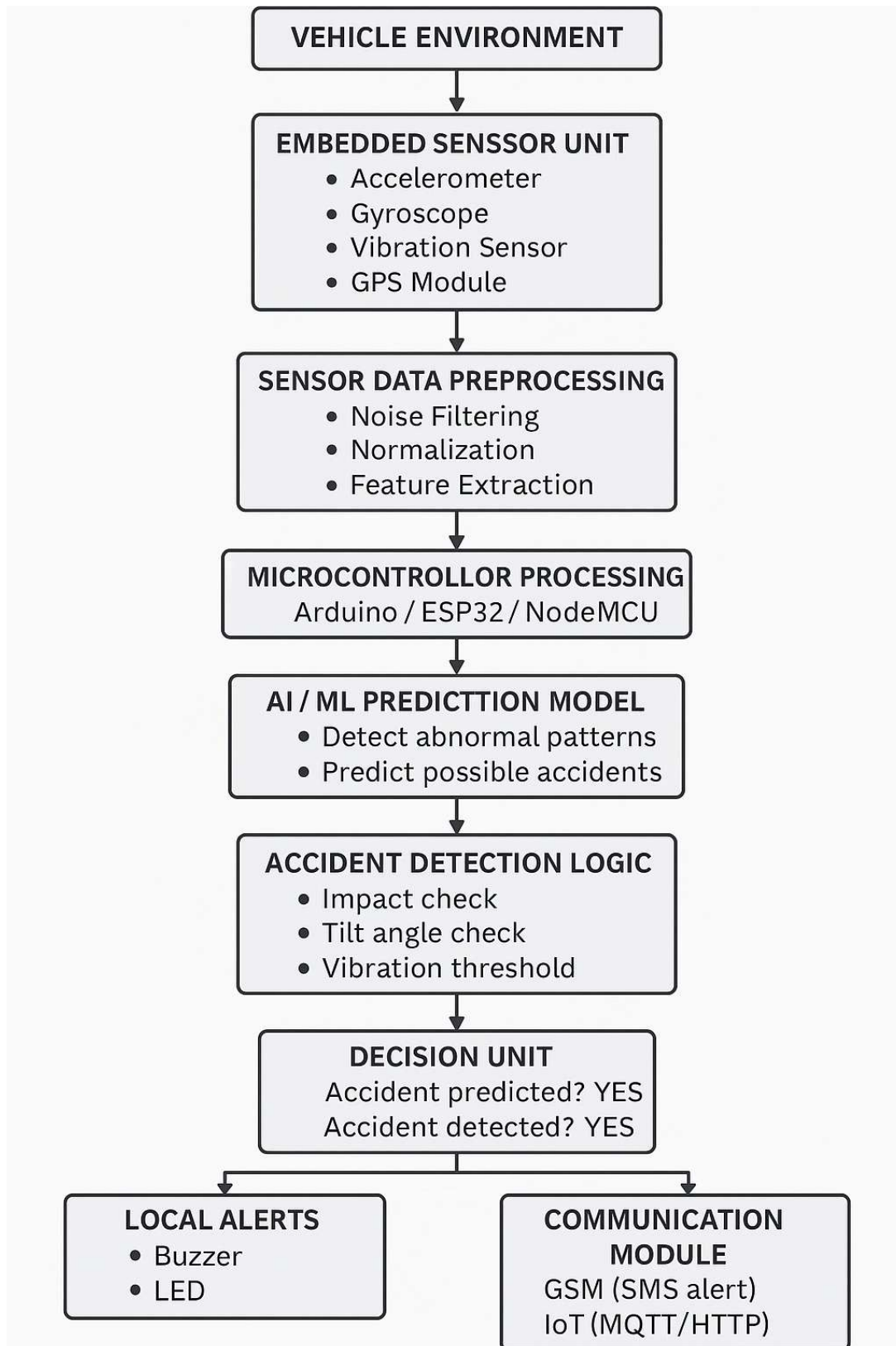


Fig1: Content diagram

4. DESIGN AND IMPLEMENTATION

3. DESIGN AND IMPLEMENTATION

4.1 Data Flow Diagram (DFD)

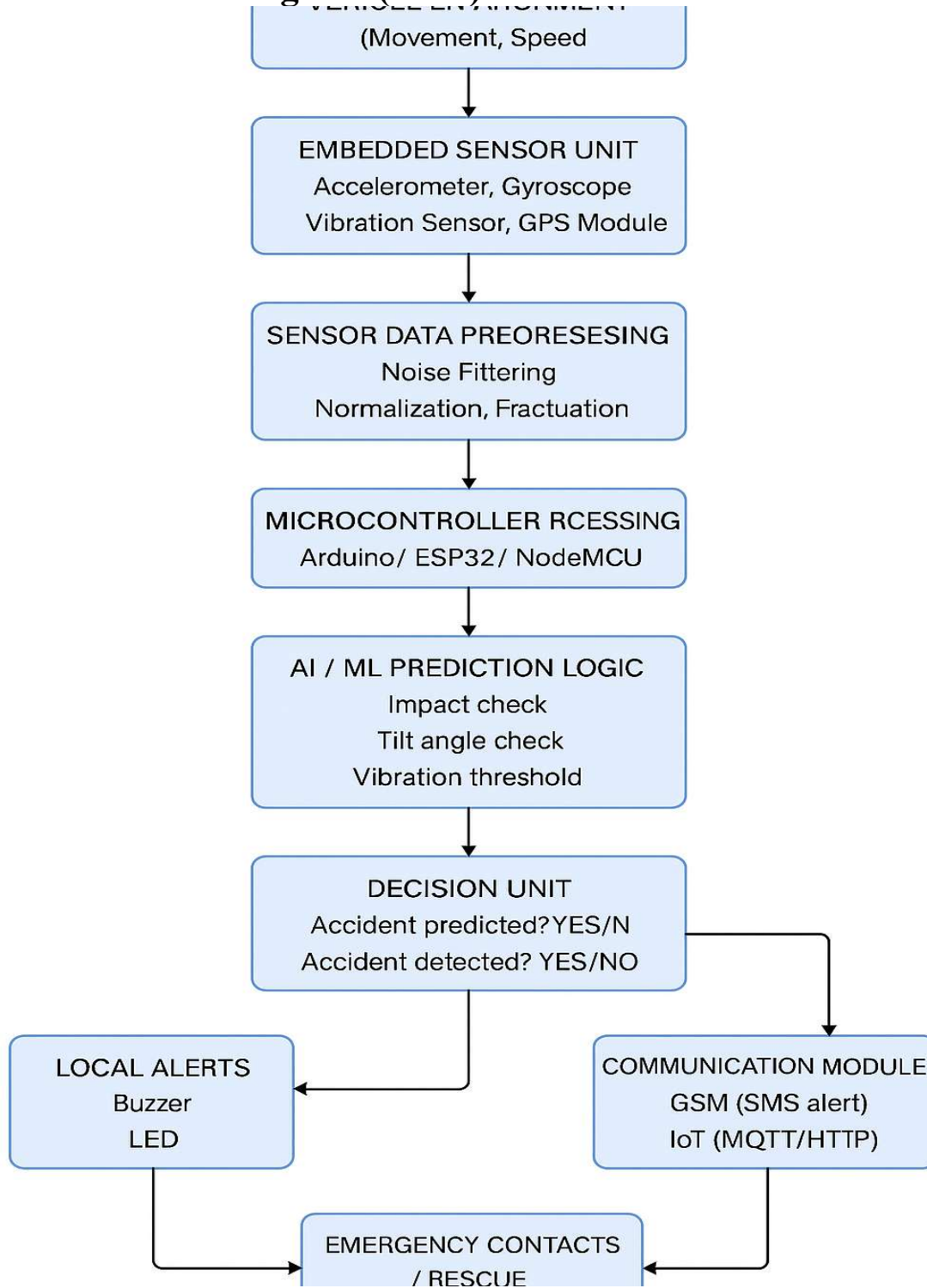


Fig2: Data flow diagram

4.2 UML Diagrams :

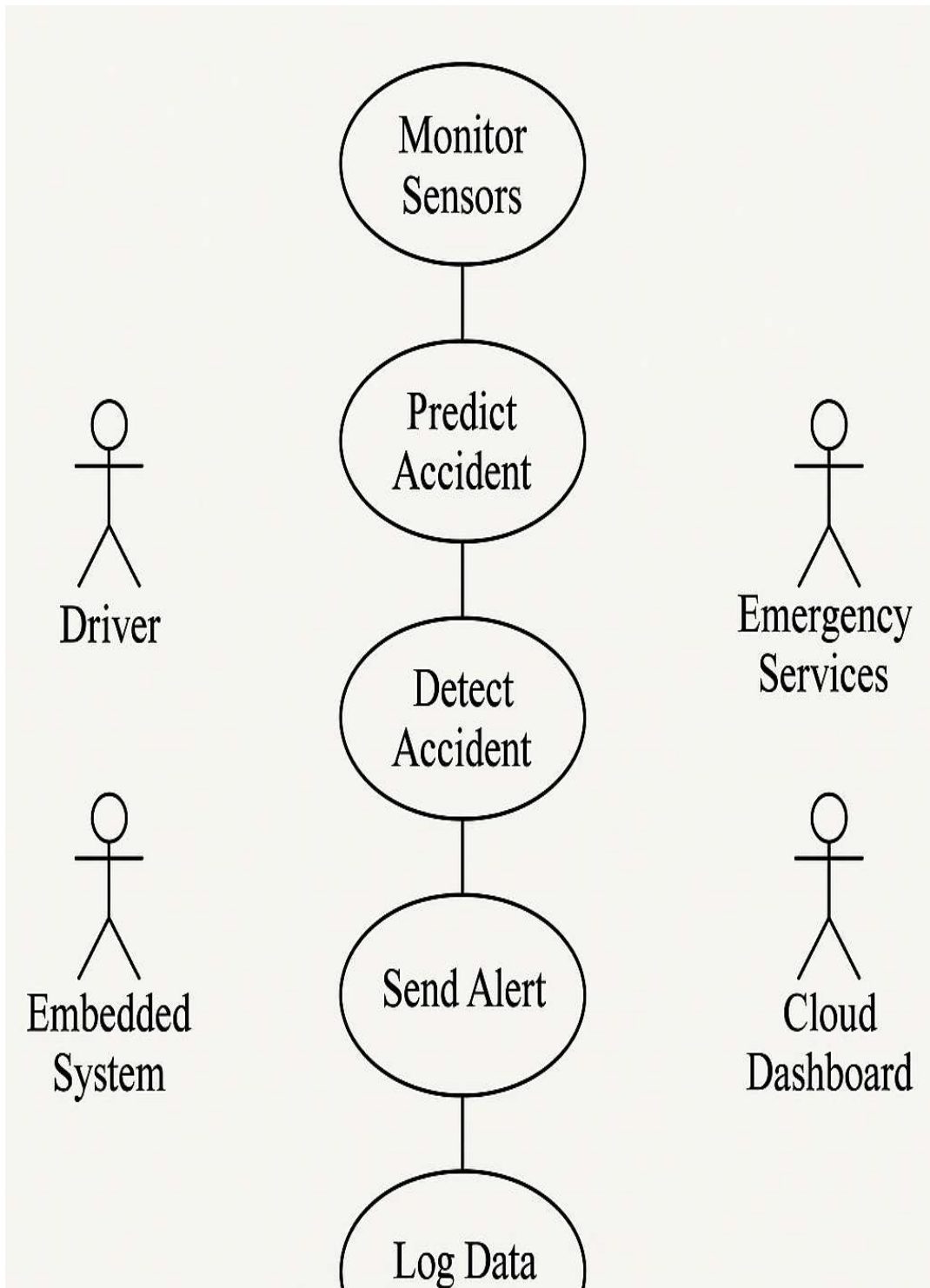


Fig3: UML diagram

4.2.1 Use Case Diagram :

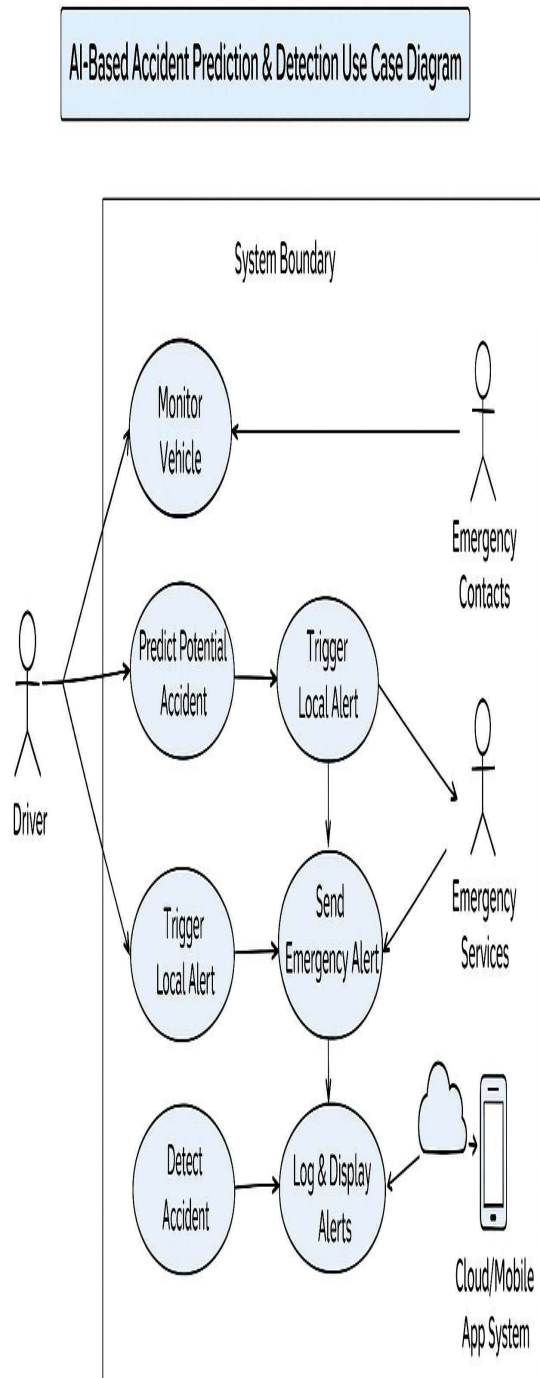


Fig4: use case diagram

4.2.2 Class Diagram:

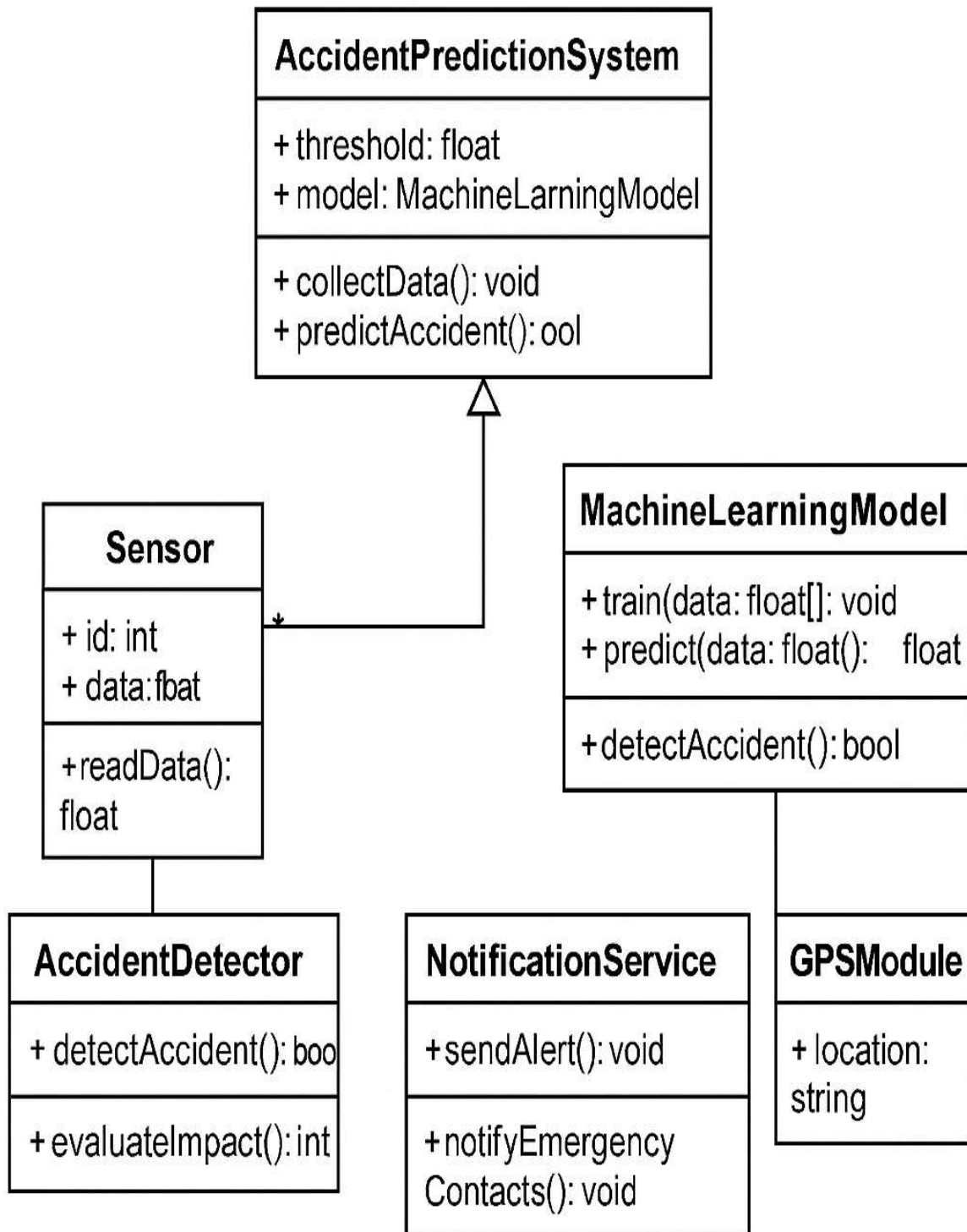


Fig5: class diagram

4.2.3 Sequence Diagram:

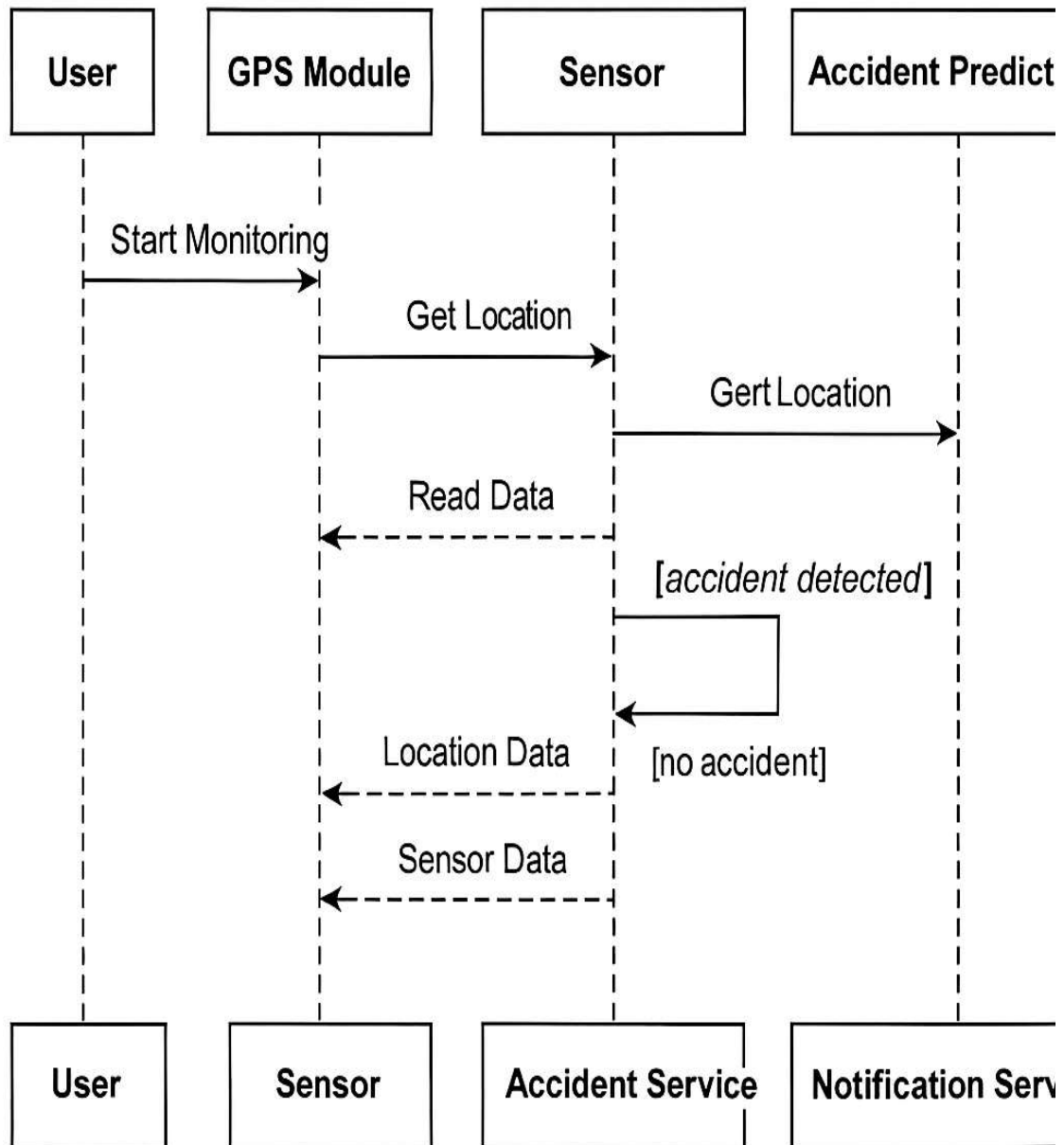


Fig6: sequence diagram

4.2.4 Activity Diagram :

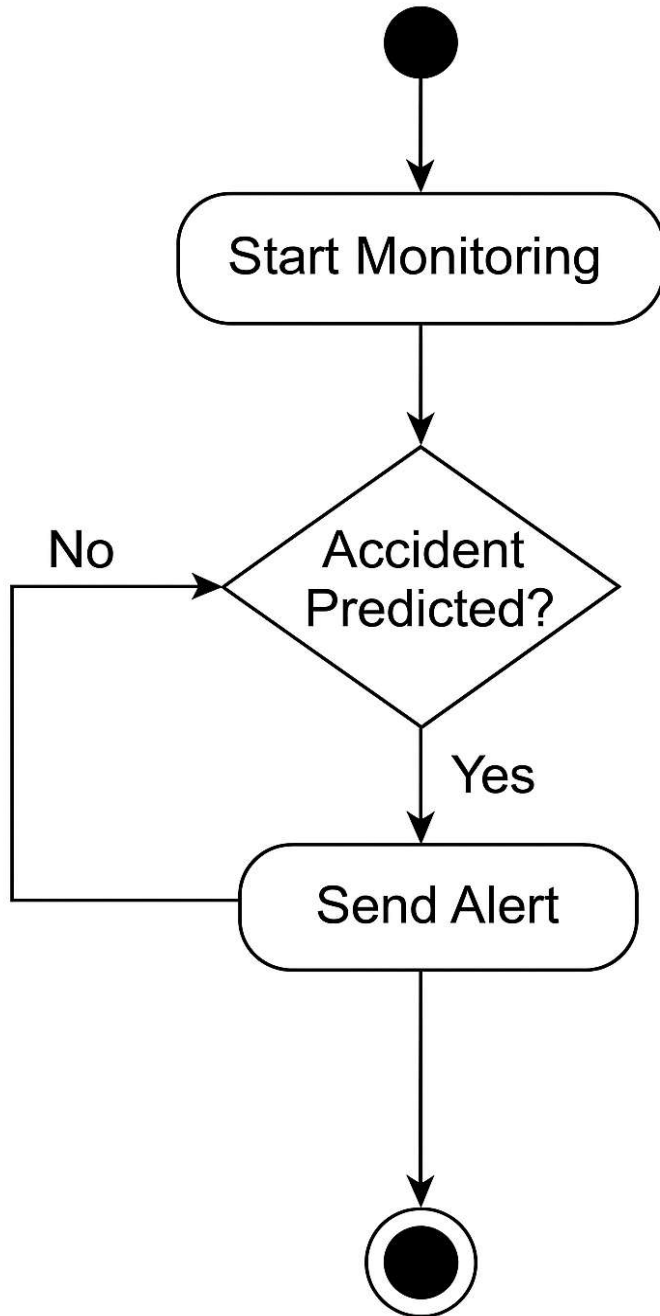


Fig7: Activity diagram

4.3 Module Design and Organization :

The module design and organization of the AI-Based Accident Prediction and Detection System follow a structured and modular approach to ensure clarity, reliability, and smooth real-time functioning. The system is divided into six well-defined modules, each performing an independent operation while working together in a systematic flow. This modular design makes the system easier to develop, understand, maintain, and upgrade.

1. Sensor Acquisition Module

This module is responsible for collecting real-time data from all the sensors installed in the vehicle. It includes the accelerometer, gyroscope, vibration sensor, and GPS unit. The sensors continuously monitor speed, acceleration, orientation, vibration, and location. The module ensures that live sensor data is captured accurately and sent to the microcontroller without delay. Since the entire system depends on this input, the module is designed to operate continuously and reliably during vehicle motion.

Points;

- Collects raw data from all sensors.
- Monitors acceleration, tilt, vibration, and location.
- Sends values continuously to the processor.
- Forms the starting point of the entire system workflow.

2. Data Preprocessing Module

The raw sensor outputs may contain noise or sudden fluctuations due to road conditions or vehicle vibrations. The preprocessing module cleans and organizes this data so it becomes suitable for further analysis. It applies filtering techniques, smoothens the input values, normalizes the data, and extracts meaningful features such as vibration intensity, tilt angle, and acceleration magnitude. This reduces errors and improves the accuracy of the machine learning model.

Points:

- Removes noise and unwanted fluctuations.
- Applies smoothing and normalization.
- Extracts features like tilt, jerk, vibration levels.
- Converts raw readings into clean feature vectors.

3. Machine Learning Prediction Module

This module contains the trained machine learning model, which is deployed on the microcontroller using TinyML or TensorFlow Lite. The model analyzes the preprocessed feature data and predicts whether the vehicle is in a normal, risky, or accident-prone condition. It operates in real time with very low latency, making it suitable for embedded systems. This module adds intelligence to the system by identifying unsafe driving patterns before an accident occurs.

Points:

- Runs the trained ML model on the microcontroller.
- Predicts accident-prone or risky behavior.
- Works with very low latency for real-time use.
- Acts as the intelligent core of the system.

4. Accident Detection Module

While prediction identifies risky behavior, the accident detection module confirms whether an actual collision has occurred. It uses a combination of machine learning output and fixed threshold rules to ensure reliability. It checks for strong impact, sudden vibration spikes, unusual tilt angles, and rapid changes in acceleration. This module prevents false alarms and ensures that only genuine accidents trigger emergency alerts.

Points:

- Validates accidents using impact and vibration.
- Combines ML output with threshold logic.
- Detects real collisions accurately.
- Reduces false positives and false triggers

5. Decision and Alert Module

This module decides what action the system should take based on the prediction and detection results. If risky behavior is detected, it activates a driver warning through a buzzer or LED indicator. If a real accident is detected, it sends a signal to the communication module for emergency alerting. This module ensures timely warnings and helps prevent accidents by notifying the driver of unsafe conditions.

Points:

- Issues buzzer/LED alerts for predicted risks.
- Activates emergency mode for detected accidents.
- Handles alert priority and decision-making.
- Acts as the bridge between detection and communication.

6. Communication and Notification Module

This final module handles communication with emergency services or predefined contacts. Using GSM, Wi-Fi, or IoT platforms, it sends alert messages along with GPS coordinates of the accident location. This helps rescue teams respond quickly. The module also ensures that alerts are retried if the network fails, making the system reliable even in poor signal areas.

Points:

- Sends GPS-based SMS/IOT alerts automatically.
- Notifies emergency contacts immediately.
- Retries messages on network failure.
- Supports GSM, Wi-Fi, or IoT communication.

4.4 Explanation of Key Functions

1. Sensor Data Collection

Collects real-time readings from accelerometer, gyroscope, vibration sensor, and GPS.

Provides continuous data needed for predicting and detecting accidents.

2. Data Filtering

Removes noise and unwanted disturbances from raw sensor data.

Ensures clean and stable values for accurate analysis

3. Feature Extraction

Converts cleaned data into useful features such as acceleration magnitude, tilt, and vibration level.

These features help the system understand vehicle behavior.

4. Machine Learning Prediction

Uses a trained TinyML / TFLite model to classify driving conditions as normal, risky, or accident-prone.

Works in real time on the microcontroller.

5. Impact Detection

Checks for sudden shocks, vibrations, or extreme tilt angles.

Confirms if a real accident has happened using threshold values.

6. Decision-Making

Decides whether to trigger a warning or emergency alert based on prediction and detection results.

7. Driver Warning

Activates a buzzer or LED to alert the driver of risky driving behavior.

Helps prevent potential accidents.

8. Emergency Alert

Automatically sends an SMS/IoT alert with GPS location to emergency contacts when an accident is confirmed

4.5 Method of Implementation

The implementation of the AI-Based Accident Prediction and Detection system is divided into distinct phases: hardware setup, data collection, data preprocessing, model training & optimization, embedded deployment, firmware development, integration & testing, and deployment/maintenance. Each phase is planned to produce concrete deliverables and enable iterative improvement.

1. Hardware Setup

1. Select and procure hardware: microcontroller (ESP32 recommended), accelerometer + gyroscope (MPU6050), vibration sensor (SW-420 or piezo), GPS module (NEO-6M), GSM module (SIM800L) or Wi-Fi module, power regulator, buzzer, LEDs, and enclosure.
2. Assemble prototype on breadboard or prototype PCB: mount sensors, wire to microcontroller, add voltage regulation and power connections.
3. Ensure secure mechanical mounting for sensors to capture stable vehicle dynamics.

2. Data Collection

1. Implement sensor drivers on a development board to sample accelerometer, gyroscope, vibration, and GPS at appropriate rates (e.g., 50–200 Hz for motion sensors; 1 Hz for GPS).
2. Create a logging pipeline to store synchronized timestamped sensor data on SD card or stream to a laptop/phone via serial/Wi-Fi.
3. Collect labeled datasets for: normal driving, harsh braking, sharp turns, skidding, collisions (simulated where safe). Ensure diversity (different vehicles, road types, speeds).
4. Maintain meta-data (vehicle type, sensor mounting position, test conditions) for each recording.

3. Data Preprocessing (Offline & Edge)

1. Offline: clean collected data — remove corrupt segments, fill missing values, and segment into windows (e.g., 1 s windows with

2. Apply filtering (moving average or Kalman) to remove high-frequency noise.
3. Extract features per window: acceleration magnitude, jerk ($\Delta \text{acc}/\Delta t$), RMS vibration, tilt angle, max/min, mean, standard deviation, GPS delta speed, etc.
4. Normalize/scale features (z-score or min-max) and save preprocessing parameters for edge use.
5. Implement a lightweight, on-device preprocessing routine that mirrors offline steps but optimized for low compute.

4. Model Training & Optimization

1. Choose candidate models (Decision Tree / Random Forest, shallow ANN, 1D-CNN for temporal features).
2. Train using balanced datasets; use augmentation or class weighting to mitigate class imbalance (accidents are rare).
3. Evaluate with metrics: recall (priority), precision, F1, confusion matrix, ROC. Prefer higher recall for safety-critical detection.
4. Optimize model for edge: pruning, quantization (post-training quantization to INT8), reduce parameters, or convert to TensorFlow Lite.
5. Test inference latency on a constrained device (emulator or target board) and iterate until latency and memory are acceptable.

5. Embedded Deployment (Firmware)

1. Integrate the quantized model into firmware using TensorFlow Lite Micro or a lightweight inference engine.
2. Implement real-time tasks:
 - Sensor sampling task (interrupt-driven or dedicated RTOS task).
 - Preprocessing & feature extraction task.
 - Inference task running model on fixed-size feature vectors.
 - Decision logic task (combine model score + threshold rules + hysteresis).
 - Communication & alert task (GSM/IoT + buzzer/LED).

6. Decision Logic & Alerts

1. Implement a two-stage decision: (A) prediction score from ML model, (B) confirmation via impact thresholds (accel spike, vibration, tilt).
2. Use hysteresis and short timers to avoid transient false positives (require condition to persist for a small window before alert).
3. For predicted risk: sound warning (buzzer/LED). For confirmed collision: automatically trigger emergency alert with GPS.

7. Communication & Backend Integration

1. Format and send emergency messages: vehicle ID, timestamp, GPS coordinates, event type. Use GSM SMS or MQTT/HTTP to cloud depending on connectivity.
2. Optionally push events to a cloud dashboard for logging and remote monitoring (enable later model retraining).
3. Implement retry logic for network failures, and local logging of unsent alerts.

8. Testing Strategy

1. Unit tests: validate sensor drivers, preprocessing functions, and inference wrapper.
2. Integration tests: full pipeline on bench (simulated events).
3. Field tests: controlled drive tests to validate real-world performance across vehicle types.
4. Performance tests: measure latency, memory usage, power draw.
5. Robustness tests: network loss, sensor disconnects, power interruption.
6. Maintain test logs and iterate model and thresholds based on false positives/negatives.

9. Power & Reliability Considerations

1. Optimize sampling rates and duty cycles to balance detection quality with power consumption.
2. Use low-power modes when vehicle idle; implement wake-on-motion if supported.
3. Add backup battery or supercap if continuous operation when vehicle off is required.

10. Deployment & Maintenance

1. Finalize PCB and automotive-grade enclosure for production-level robustness.
2. Implement secure firmware update mechanism (OTA for ESP32) for future improvements.
3. Provide clear user manual for installation, calibration, and emergency contact configuration.
4. Collect anonymized event logs (with consent) to retrain and improve models.

4.6 output screens

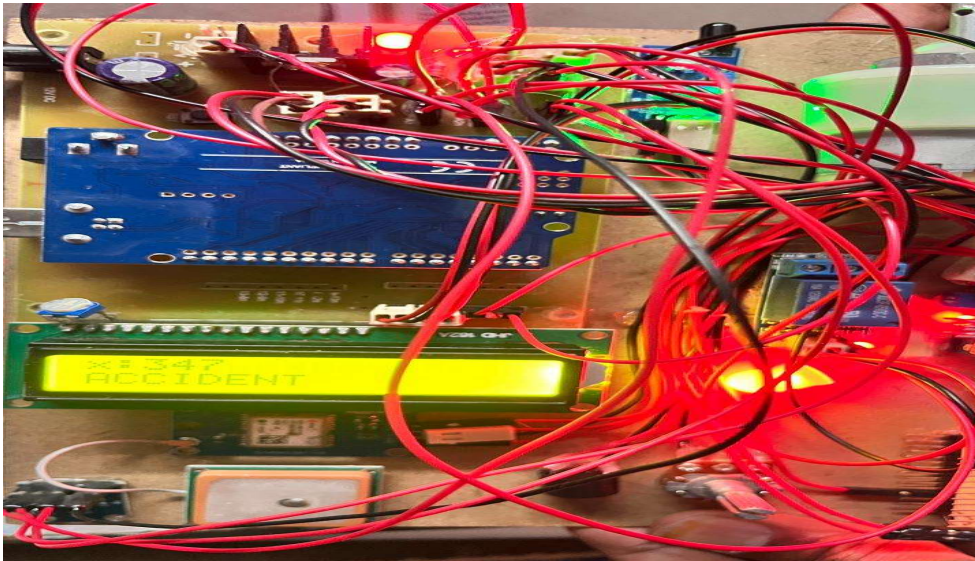


Fig 8

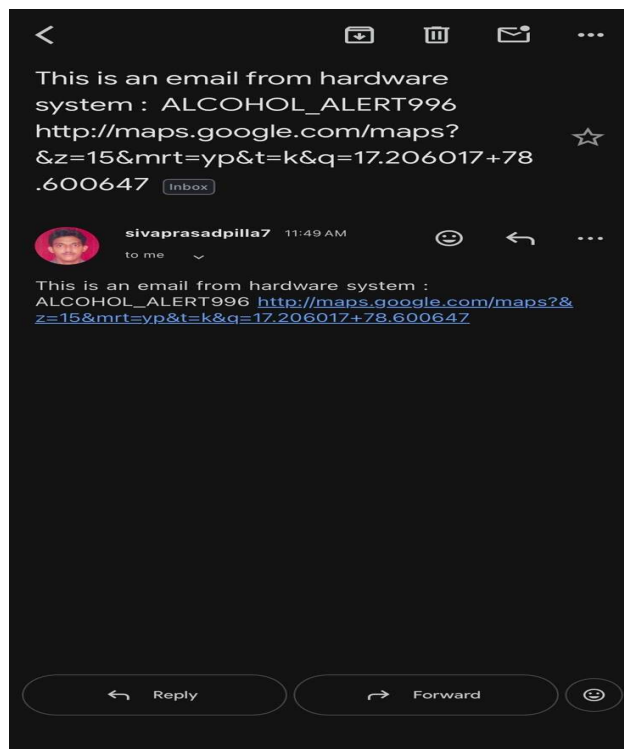


Fig 9

4.7 Group photo with prototype



5. RESULT ANALYSIS

2. RESULT ANALYSIS

Introduction

2.1 Design of Test cases and scenarios

2.1.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

2.1.2 Integration testing

Integration tests are designed to test integrated software components to determine if they run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

2.1.3 Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : Identified classes of valid input must be accepted.

Invalid Input : Identified classes of invalid input must be rejected.

Functions : Identified functions must be exercised.

Output : Identified classes of application outputs must be exercised.

Systems/ Procedures: Interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

2.1.4 System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

1 Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases

2 Test strategy and approach

Field testing will be performed manually and functional tests will be written in detail.

3 Test objectives

1. All field entries must work properly.
2. Pages must be activated from the identified link.
3. The entry screen, messages and responses must not be delayed.

4 Features to be tested

1. Verify that the entries are of the correct format
2. No duplicate entries should be allowed
3. All links should take the user to the correct page.

5 Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

5.2 Test Results

All the test cases mentioned above passed successfully. No defects encountered.

5.2.1 Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

5.2.2 Results

All the test cases mentioned above passed successfully. No defects encountered.

6. CONCLUSION

6.CONCLUSION

The AI-Based Accident Prediction and Detection System using embedded sensors provides an effective and intelligent solution for improving road safety and reducing the delay in emergency response. The project successfully integrates embedded hardware, machine learning techniques, and real-time communication modules to predict risky driving conditions and detect accidents with high accuracy. By continuously monitoring vehicle acceleration, vibration patterns, tilt angles, and location data, the system ensures that abnormal or dangerous driving behaviors are identified at an early stage, thereby helping prevent potential accidents.

The implementation of lightweight machine learning models on microcontrollers demonstrates that advanced AI functionality can be deployed even on low-power embedded devices. The system's modular design, consisting of sensor acquisition, data preprocessing, ML prediction, accident detection, and communication modules, ensures efficient operation, easy maintenance, and scalability for future enhancements. The use of GSM and GPS technologies allows timely transmission of accident alerts and precise location details to emergency contacts, significantly reducing the response time during critical situations when the driver may be unconscious or unable to call for help.

This project also highlights the importance of integrating AI with embedded systems to solve real-world problems. The successful deployment of the model in real-time conditions shows that such solutions can be practically implemented in vehicles to save lives. With further improvements—such as cloud-based data analytics, advanced deep-learning models, and integration with vehicular networks—the system can be transformed into a more robust..

7. REFERENCE

7.REFERENCES

1. **Rajesh, A., & Suresh, K.**, “Accident Detection System Using Accelerometer and GPS Module,” *International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering*, vol. 5, no. 7, pp. 45–50, 2020.
2. **Sharma, P., & Verma, N.**, “Machine Learning-Based Vehicle Accident Prediction System,” *International Journal of Computer Applications*, vol. 182, no. 45, pp. 22–28, 2021.
3. **Patel, R., & Singh, V.**, “Real-Time Accident Detection Using IoT and GSM Technologies,” *International Journal of Engineering Research & Technology*, vol. 9, no. 11, pp. 1–5, 2020.
4. **N. Dey, A. Ashour, & S. Chakraborty**, *Smart Sensors for Real-Time Monitoring*, Springer, 2020.
5. **Kumar, S., & Mehta, P.**, “Application of Accelerometer and Gyroscope for Vehicle Collision Detection,” *IEEE Sensors Journal*, vol. 18, no. 10, pp. 4125–4132, 2019.
6. **TensorFlow Lite Micro**, “TinyML on Embedded Devices,” Available: <https://www.tensorflow.org/lite/micro> (Accessed: 2025).
7. **Gupta, M., & Rathore, R.**, “IoT-Based Smart Vehicle Monitoring System,” *International Journal of Innovative Research in Science, Engineering and Technology*, vol. 8, no. 3, pp. 234–239, 2021.
8. **MPU-6050** Accelerometer/Gyroscope Datasheet, InvenSense Inc., 2020.
9. **SIM800L GSM/GPRS** Module Datasheet, SIMCom Wireless Solutions, 2021.
10. **Google Developers**, “Machine Learning Crash Course,” Available: <https://developers.google.com/machine-learning> (Accessed: 2025).

ANNEXURE

```

#include <LiquidCrystal.h>
const int rs = 13, en = 12, d4 = 11, d5 = 10, d6 = 9, d7 = 8;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

#include <SoftwareSerial.h>
#include <TinyGPS++.h>
TinyGPSPlus gps;
SoftwareSerial gpsSerial(6, 7); // RX, TX (GPS TX -> 4, GPS
RX -> 3)

float latitude = 17.206017271102375;//17.206017271102375,
78.60064607658057
float longitude = 78.60064607658057;
int buzzer=A1;
int fire=3;
int m1=5;
int m2=4;
int motor=A2;
void setup()
{
  pinMode(fire,INPUT);
  pinMode(buzzer,OUTPUT);digitalWrite(buzzer,HIGH);
  pinMode(motor,OUTPUT);digitalWrite(motor,HIGH);
  pinMode(m1,OUTPUT);pinMode(m2,OUTPUT);
  digitalWrite(m1,LOW);digitalWrite(m2,LOW);
  Serial.begin(9600);
  gpsSerial.begin(9600);
  //Serial.println("Starting GPS...");
  lcd.begin(16, 2);

```

```

    lcd.clear();lcd.print("WELCOME");delay(2000);
    digitalWrite(m1,HIGH);digitalWrite(m2,LOW);
}
void beep()
{
    digitalWrite(motor,HIGH);delay(1000);
    digitalWrite(m1,LOW);digitalWrite(m2,HIGH); delay(1000);
    digitalWrite(buzzer,LOW);delay(500);digitalWrite(buzzer,HIGH);delay(200);
    digitalWrite(buzzer,LOW);delay(500);digitalWrite(buzzer,HIGH);delay(200);
}
void loop()
{
    back:
    String data="";
    // Read data from GPS
    while (gpsSerial.available() > 0) {
        gps.encode(gpsSerial.read());
    }

    // Check if GPS has a valid fix
    if (gps.location.isValid() && gps.location.isUpdated()) {
        latitude = gps.location.lat();
        longitude = gps.location.lng();
    }
    else {
        // No GPS signal → default 0.00
        latitude = 17.206017271102375;
        longitude = 78.60064607658057;
    }
}

```

```

}

//lcd.clear(); lcd.print("Lati:");lcd.print(latitude, 6);
//lcd.setCursor(0,1);lcd.print("Longi:");lcd.print(longitude,
6);delay(100);
String location="
http://maps.google.com/maps?&z=15&mrt=yp&t=k&q="+Stri
ng(latitude,6)+"+"+String(longitude,6);
delay(1000);

int speed=analogRead(A0);delay(1);
speed=map(speed, 0, 1023, 0, 100);

if(speed<=0)
{
digitalWrite(motor,HIGH);delay(1000);
lcd.clear();lcd.print("VEHICLE");delay(1);
lcd.setCursor(0,1);lcd.print("STOP");delay(1000);goto back;
}
lcd.clear();lcd.print("Sp:");lcd.print(speed);delay(100);

int gas=analogRead(A3);delay(10);
lcd.setCursor(0,1);lcd.print("Al:");lcd.print(gas);delay(1000);

int x=analogRead(A4);delay(10);
int y=analogRead(A5);delay(10);
lcd.clear();
lcd.setCursor(0,0);lcd.print("x:");lcd.print(x);lcd.print(" ");
lcd.setCursor(0,1);lcd.print(" y:");lcd.print(y);lcd.print("
");delay(100);

```

```

int fireval=digitalRead(fire);delay(10);
if(fireval==LOW)
{
  lcd.setCursor(0,1);lcd.print("FIRE ALERT
");delay(1000);beep();beep();
  data="FIRE_ALERT";
  delay(1000);lcd.clear();lcd.print("Sending Data..");
  data=data+location;delay(1000);Serial.print(data);delay(1000);
  goto back;
}


if(x>370 || x<300 || y>370 || y<300)
{
  lcd.setCursor(0,1);lcd.print("ACCIDENT ALERT
");delay(1000);beep();beep();
  data="ACCIDENT_ALERT";
  delay(1000);lcd.clear();lcd.print("Sending Data..");
  data=data+location;delay(1000);Serial.print(data);delay(1000);


  goto back;
}
if(gas>=600)
{
  lcd.clear();lcd.print("ALCOHOL ALERT
");delay(1000);beep();beep();
  data="ALCOHOL_ALERT"+String(gas);
  delay(1000);lcd.clear();lcd.print("Sending Data..");
  data=data+location;delay(1000);Serial.print(data);delay(1000);

```

```
goto back;  
}
```

```
if(speed>=70)  
{  
lcd.clear();lcd.print("OVER SPEED ALERT  
");delay(1000);beep();beep();  
data="OVER_SPEED_ALERT:"+String(speed);  
delay(1000);lcd.clear();lcd.print("Sending Data..");  
data=data+location;delay(1000);Serial.print(data);delay(1000);  
  
goto back;  
}  
digitalWrite(motor,LOW);digitalWrite(m1,HIGH);digitalWrite  
(m2,LOW);  
}
```